Names: Ishimwe Pacis Hanyurwimfura
Student Number: 3787234

Link to Excel with results running time for different processes, modes, function

Link to Github Repository

Assignment 2 Report

A. Design Summary

a. Adaptive Integration Method

The program computes integrals on the interval [0,1] using Adaptive Simpson's Rule. For an interval [a,b], it first computes one Simpson estimate on the whole interval, then computes a refined estimate by splitting the interval at the midpoint and applying Simpson's Rule to the two halves. The local error is estimated by the difference between the refined and coarse estimates. If this error is within the tolerance, the interval is accepted; otherwise, the interval is split again recursively. A safety limit MAX_DEPTH = 50 is used to prevent unbounded recursion. The implementation also halves the tolerance when subdividing into child intervals which is applicable for all 3 modes.

b. Task Granularity Choice (K and Rationale)

In the static version, the interval is partitioned into **K** equal coarse intervals and distributed in fixed blocks across ranks. In the initial implementation, **K** was chosen as the number of processes for simplicity. Later, a larger **K** was used for the oscillatory function (function 2) to avoid overly coarse starting intervals and improve correctness.

In the dynamic version, each task stores an interval's start and end points, its tolerance, and its current depth in the form (a, b, tol, depth), and the initial number of tasks used **K** = 4 * the number of workers so there would be enough tasks available early in the run. If **K** were too small, some workers could finish quickly and wait and If **K** were too large, task management overhead would increase. Using 4 * workers gave a reasonable balance between keeping workers busy and avoiding too much communication overhead.

c. Master/Worker Communication Design

Mode 2 uses rank 0 as the master and all other ranks as workers. Communication is tag-based:

- **TAG_READY**: a worker tells the master it is idle and ready for work
- **TAG_TASK:** the master sends one task to a worker
- **TAG_NEW_TASK:** a worker returns a newly created task to the master after splitting
- **TAG_STOP:** the master tells a worker to terminate
- **TAG_REPORT:** a worker sends its final subtotal, accepted interval count, and active compute time back to the master

Workers process tasks using a local task stack. If a task is accepted, its contribution is added to the worker's local subtotal. If it must be split, the worker keeps one child locally and sends the other child back to the master as a new task. This design supports dynamic load balancing without forcing the master to perform numerical work itself

d. Termination Strategy

Termination in the dynamic version is based on two conditions:

1. The master's task pool is empty
2. No worker is still active

This avoided stopping too early, since a worker that is still active may split a task and generate more work. The master tracks worker activity and only sends TAG_STOP once the task pool is empty and no worker remains active and then after that, it receives final reports from all workers and combines their subtotals into the final integral estimate.

B. Results Tables

The whole results can be found in this Google sheets all the 10 runs per each mode function and processor being (2,4,8) and removing the first run .
P.S My laptop couldn't manage over 8 processes.

Below are the runtime summary :

Function 1, tolerance = 1e-6

Runtime Summary

Processes	Mode 0 Avg Runtime (s)	Mode 1 Avg Runtime (s)	Mode 2 Avg Runtime (s)	Correctness Note
2	0.000006	0.000026	0.001478	Within bound
4	0.000006	0.000080	0.001678	Within bound
8	0.000006	0.000087	0.000170	Within bound

Speedup and Efficiency

Processes	Mode 1 Speedup / Efficiency	Mode 2 Speedup / Efficiency	Serial Avg Runtime (s)
2	0.220779 / 0.110390	0.003833 / 0.001916	0.000006
4	0.070833 / 0.017708	0.003376 / 0.000844	0.000006
8	0.064885 / 0.008111	0.033312 / 0.004164	0.000006

Function 1, tolerance = 1e-8

Runtime Summary

Processes	Mode 0 Avg Runtime (s)	Mode 1 Avg Runtime (s)	Mode 2 Avg Runtime (s)	Correctness Note
2	0.000016	0.000058	0.001469	Within bound
4	0.000016	0.000075	0.000989	Within bound
8	0.000016	0.000102	0.002067	Within bound

Speedup and Efficiency

Processes	Mode 1 Speedup / Efficiency	Mode 2 Speedup / Efficiency	Serial Avg Runtime (s)
2	0.271154 / 0.135577	0.010668 / 0.005334	0.000016
4	0.207658 / 0.051915	0.015834 / 0.003958	0.000016
8	0.154098 / 0.019262	0.007580 / 0.000948	0.000016

Function 2, tolerance = 1e-8

Runtime Summary

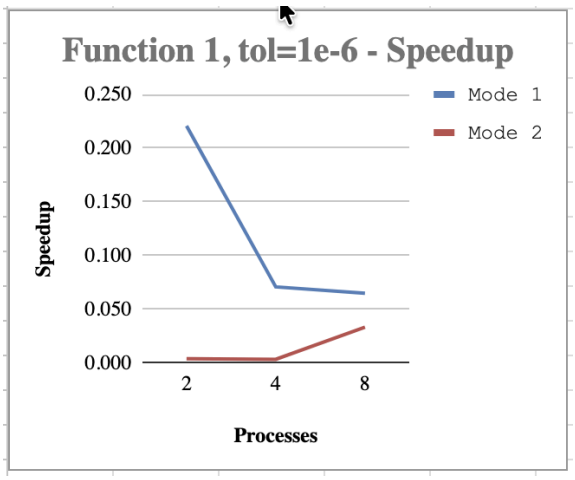
Processes	Mode 0 Avg Runtime (s)	Mode 1 Avg Runtime (s)	Mode 2 Avg Runtime (s)	Correctness Note
2	0.000948	0.000634	0.005023	Within bound
4	0.000948	0.000340	0.003092	Within bound
8	0.000948	0.000221	0.002848	Within bound

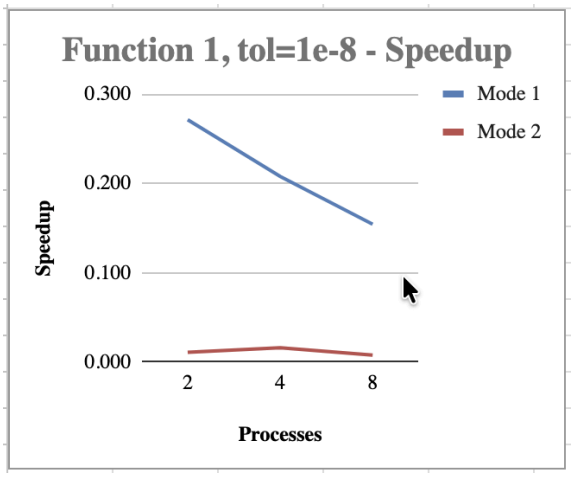
Speedup and Efficiency

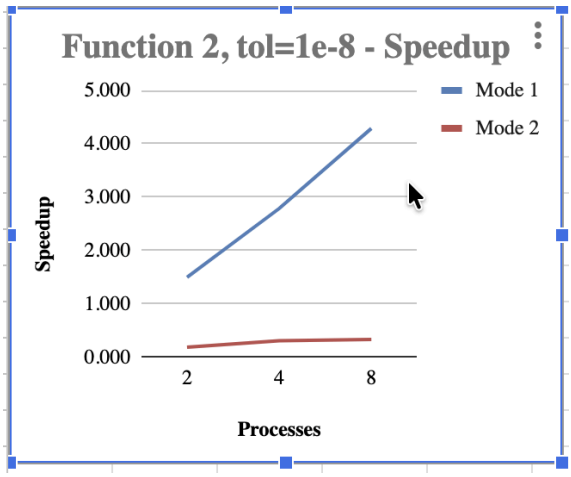
Processes	Mode 1 Speedup / Efficiency	Mode 2 Speedup / Efficiency	Serial Avg Runtime (s)	Comment
2	1.497019 / 0.748509	0.188829 / 0.094414	0.000948	
4	2.787720 / 0.696930	0.306730 / 0.076683	0.000948	
8	4.287293 / 0.535912	0.333034 / 0.041629	0.000948	

C. Plots

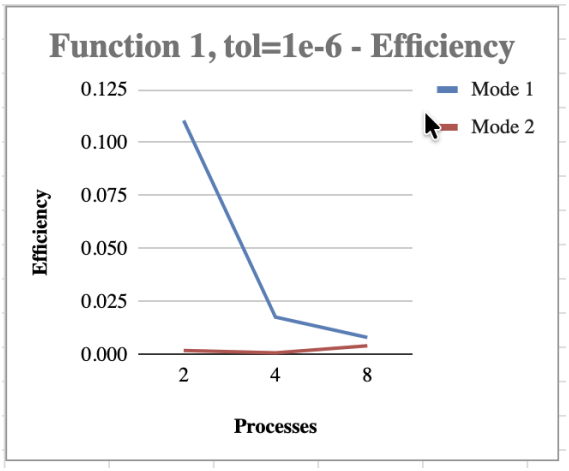
a. SpeedUp vs Number of Processes

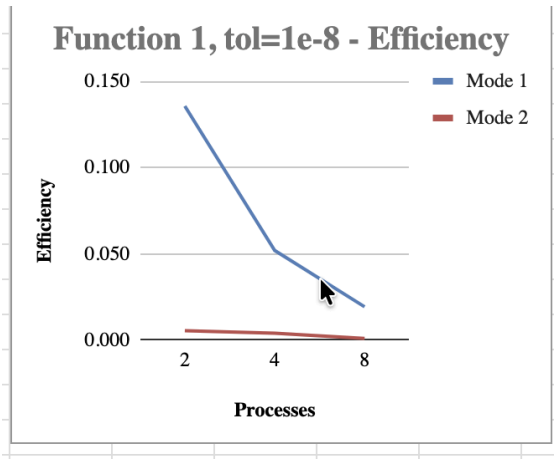


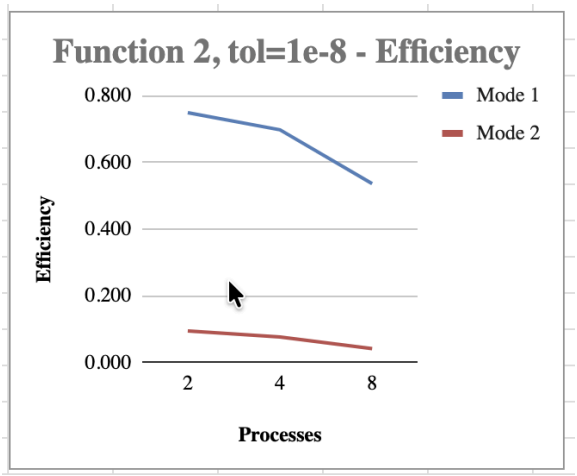




b. Efficiency Vs Number of Processes







D. Discussion

a. Why does static decomposition perform poorly for adaptive workloads?
ANS/ Static decomposition assigns each rank a fixed part of the interval prior to knowing the actual quantity of work while In adaptive integration some subintervals need much more refinement than others, so this can result in one rank ending up with a much heavier block while others finish early and be idle.

b. How does dynamic task creation improve load balance?
   ANS/ Dynamic scheduling improves balance by allowing workers to request work when they become idle, and newly formed subproblems are returned to the master for redistribution.
c. What is the cost of dynamic scheduling in MPI?
   ANS/ The main cost is additional communication and coordination. Before the computation can be completed in dynamic mode, more messages (READY, TASK, NEW_TASK, STOP, and REPORT) are required, more management at the master, and more synchronization before the computation can finish.
d. Which function benefited most from dynamic scheduling, and why?
   ANS/ Function 1 benefited the most from dynamic scheduling because its workload was more dynamic. In the static version, some ranks had significantly bigger intervals than others, therefore the load was not evenly distributed. In the dynamic version, tasks were distributed more fairly among workers. Function 2 benefited less because we had already modified its static version with a finer starting partition, resulting in a more balanced workload from the start.

Note: Looking at the diagrams above and result, we can say that dynamic scheduling is most useful when the adaptive workload is uneven, but its communication overhead can outweigh that benefit on small problem sizes or when static partitioning is already fine enough.